



UNIVERSIDAD TÉCNICA FEDERICO SANTA MARIA

DEPARTAMENTO DE INDÚSTRIAS

SANTIAGO – CHILE



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

*LABORATORIO: AUTOMATIZACIÓN DE PROCESOS CON
N8N*

CATALINA JOFRE BRAVIN

<https://github.com/CataJB/ICN292-Lab3-Jofre-Catalina->

PROFESOR JOSE GONZALEZ

AYUDANTE CATALINA BODALEO

21 de septiembre de 2026



RESUMEN EJECUTIVO

El presente informe muestra el diseño, la implementación y la evaluación técnica del sistema automatizado solicitado por la empresa AndesHogar SpA para procesar las solicitudes de devolución, dicho sistema fue realizado en la plataforma n8n, para esto se utilizaron los valores $D=14$ y $U=37000$ los cuales son derivados de utilizar $S=557$.

El principal objetivo de este laboratorio es conseguir la optimización respecto a la atención de devoluciones mediante un sistema de clasificación automática de solicitudes basándose en reglas predefinidas, la conversión monetaria a UF a través de servicios externos y la generación de registro de auditoria para la consolidación de métricas operativas.

El presente laboratorio conllevó la creación/construcción de un flujo triage mediante webhooks, reglas ya establecidas y el proceso de registro de los resultados, sumado a esto se evaluó ciertos posibles fallos que podrían llegar a ocurrir, esto a través de evaluaciones de condiciones de frontera, validación de la arquitectura ante escenarios de falla y entradas no validas, todo esto se fue documentando para su respectivo análisis. Los resultados obtenidos de estas pruebas conllevan a concluir y confirmar que la automatización reduce efectivamente los tiempos de respuesta, consigue garantizar una aplicación consistente de las políticas y reglas solicitadas por la empresa y permite un manejo seguro de excepciones.

PARTE A

Al momento de realizar la creación del flujo Triage se utiliza el nodo Webhook como nodo inicial el cual recibe las solicitudes HTTP POST desde un flujo paralelo denominado “emisor”, posterior a esto el flujo evalúa las reglas otorgadas por la empresa mediante el nodo Switch, el cual según lo solicitado por las reglas lleva al producto a su respectiva clasificación, posterior a este nodo procede a calcular el valor de la UF, basándose en la información entrega por la página mindicador.cl y estructura el flujo resultante en el registro final.

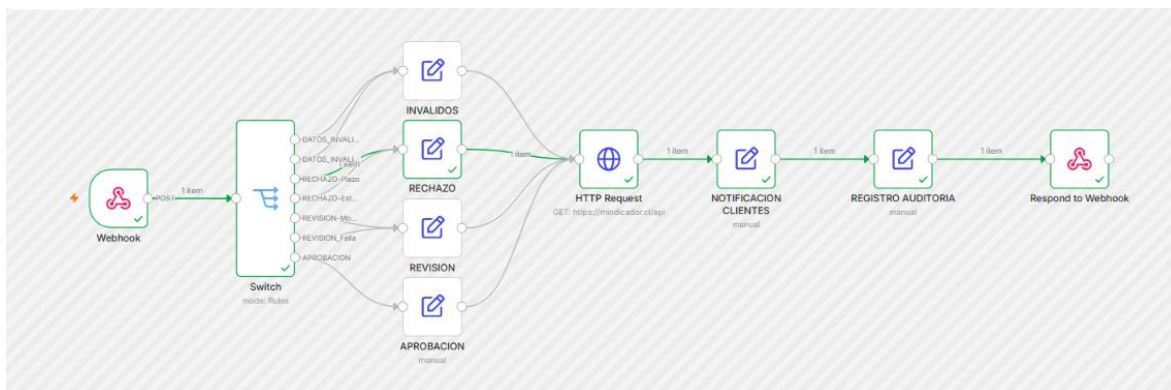


Ilustración 1: Flujo Triage camino RECHAZO

OUTPUT 1 item										
id_solicitud	sku	monto	ruta	motivo	U_aplicado	D_aplicado	valor_uf	monto_uf	dias_desde_compra	estado_produ
DEV-1125	TE-330	34990	RECHAZO	excede plazo permitido o el producto esta dañado	37000	14	40999.93	0.8534	22	sin_abrir

Ilustración 2: Registro resultante rechazado por exceder plazo

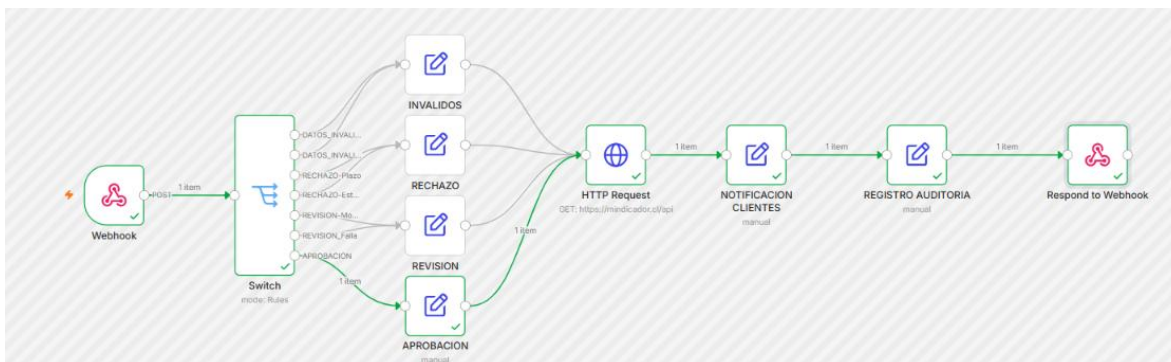


Ilustración 3: Flujo Triage camino APROBACION

id_solicitud	sku	monto	ruta	motivo	U_aplicado	D_aplicado	valor_uf	monto_uf	dias_desde_compra	estado_p
DEV-1115	AU-210	19990	APROBACION	cumple con la política de devolución	37000	14	40999.93	0.4876	3	sin_abrir



Ilustración 4: Registro resultante

PARTE C

Se expuso el flujo principal a variados problemas que se podrían dar en el día a día.

ID de solicitud vacío:

En el flujo emisor se envió el JSON modificado dejando la sección “id_solicitud” vacía, lo esperado del programa es que el nodo switch evalúe el hecho de que este valor este vacío (string is emty) y lleve la solicitud por la ruta INVALIDO_CAMPO al ejecutar el programa es justamente lo mencionado lo que sucede.

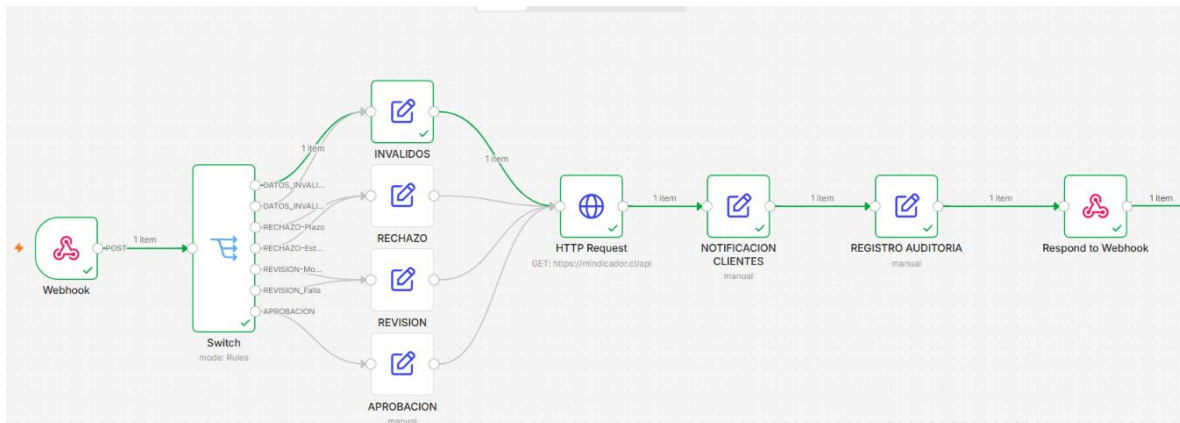


Ilustración 5: El flujo toma la ruta esperada.

Monto Negativo:

Dentro del flujo emisor se modifico el JSON reescribiendo el valor de un producto a negativo, se espera que se realice un desvío controlado hacia la rama de datos inválidos, esto debido a la regla numero 2, lo que ocurre internamente en el switch es que este evalúa la función $\text{Number} < 0$ y al cumplirse esta regla la dirige por la ruta de DATOS_INVALIDOS-Monto la que después se conecta con el nodo INVALIDOS.

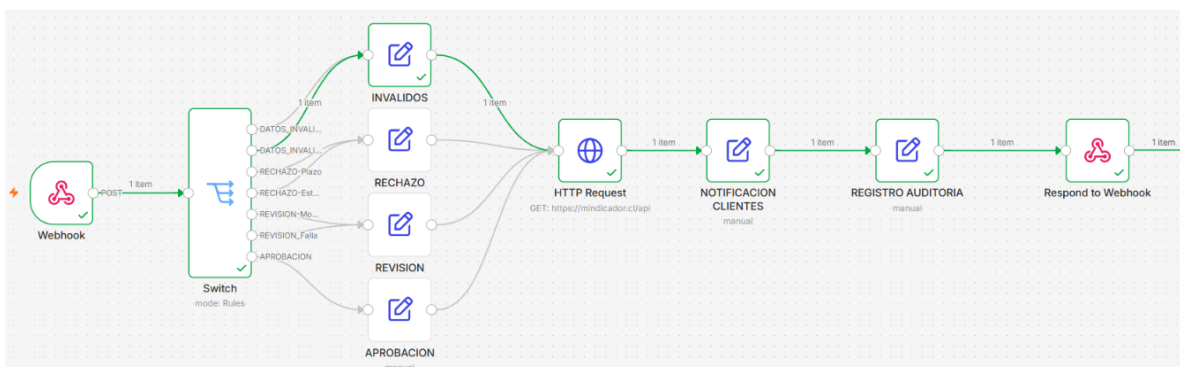


Ilustración 6: El producto toma la ruta esperada.

Monto en Palabras:

Se modifico el JSON utilizado en el flujo emisor cambiando el valor de monto de un valor numérico a un texto (20000 a veinte mil), lo esperado al realizar este cambio es que el flujo presente un error en su ejecución y que esta no se pueda realizar de manera total, esto debido a la función toNumber() la cual no procesa texto lo que genera el error y la posterior detención de la ejecución.

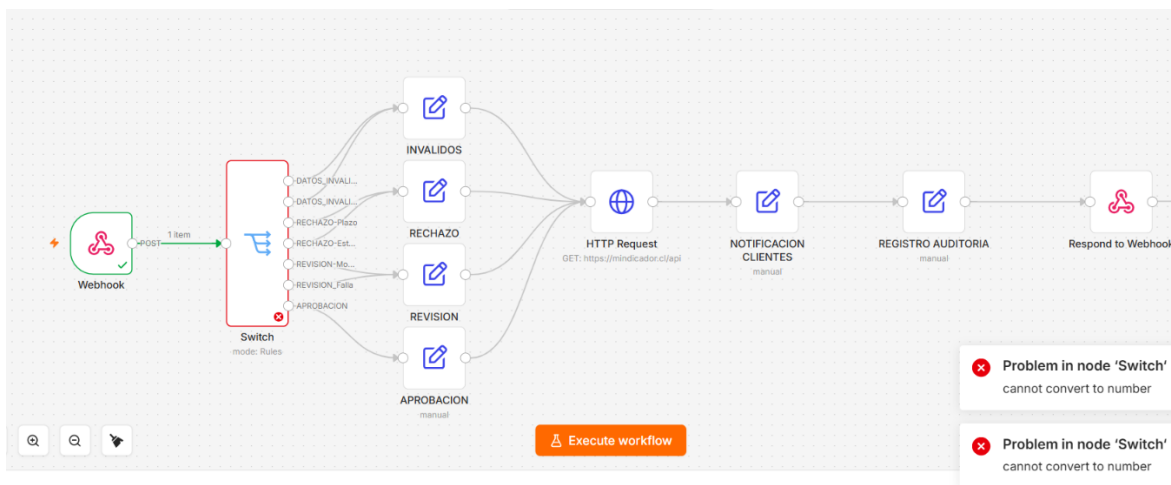
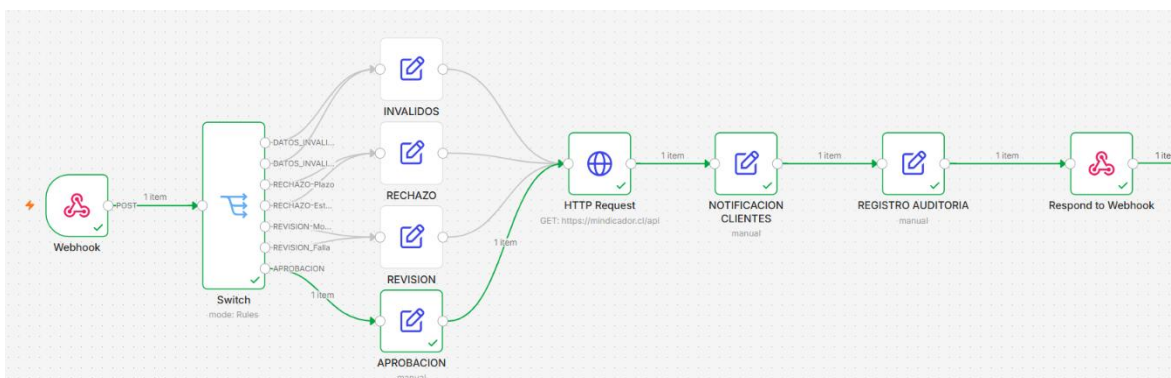


Ilustración 7: El programa tira error debido a que la función no se puede efectuar.

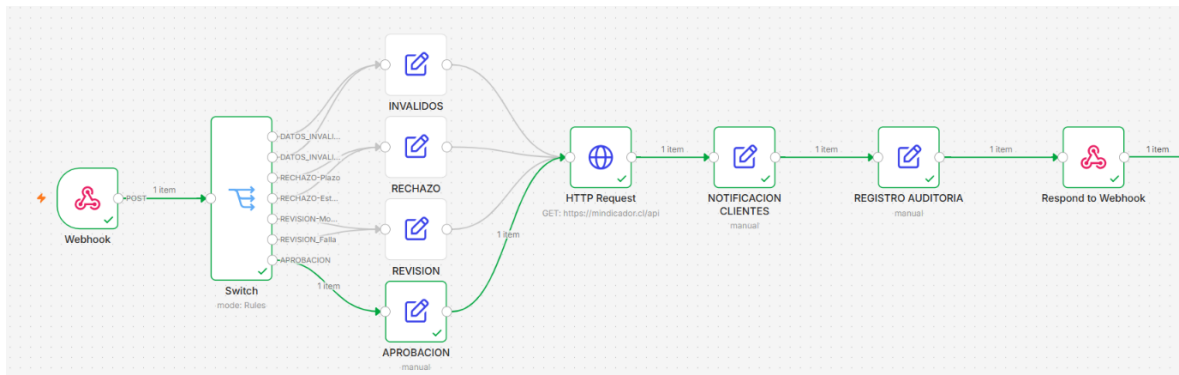
Casos Limites:

Debido a que en el listado entregado no se presentaban los casos limites, estos se agregaron de manera manual cambiando el valor del monto a 37000 y el de días_desde_compra a 14 ya que estos son los valores resultantes al utilizar el valor $S = 557$.

Al modificar el valor del monto por el valor limite (37000) el resultado dentro del flujo fue que este tomo la ruta de APROBACION ya que la regla a seguir es que si $\text{monto} > U$ (37000) este se ira por la línea de REVISION-Monto pero como en este caso esta proposición es falsa, la solicitud no se envía a revisión y se clasifica automáticamente como aprobada.



Al momento de realizar el mismo cambio, pero esta vez con el valor de días_desde_compra ocurre lo mismo ya que la función a evaluar es $\text{días} > D$ (14) por lo que al ser esta proposición falsa el flujo toma de misma forma la ruta de APROBACION.

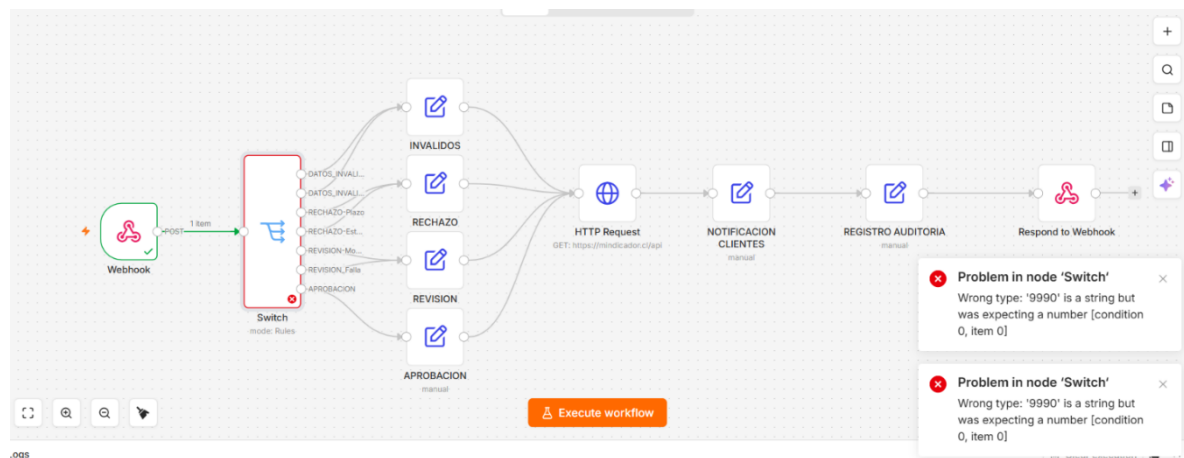


Simulación de Error de Tipo:

Para conseguir el error solicitado se omitió la función `toNumber()` dentro del nodo Switch específicamente en la condición de monto.

Esto conlleva a que el flujo no se pueda ejecutar ya que dará error, esto debido a que al momento de ejecutar tanto el flujo triage como el flujo emisor las peticiones HTTP enviadas desde el emisor a través del nodo Webhook transmiten los datos en formato texto, por lo que cuando la función dentro del Switch exige una evaluación numérica sin la mediación de una función que logre pasar este texto a un valor numérico, la evaluación realizada por el programa detiene su ejecución para así prevenir resultados erróneos.

En el caso que la naturaleza del flujo forzara la comparación de variables, la evaluación pasaría a efectuarse mediante un orden alfabético, bajo el criterio descrito anteriormente el texto ejecutado “9990” es evaluado como mayor que “30000”, debido al valor asignado al primer carácter, lo que conlleva a derivar solicitudes de menor valor a la rama de revisión, lo cual es un error.





Configuración On Error:

Se utilizó Stop Workflow en el Switch/ Edit Fields debido a que si la lógica de negocio o formato de la función falla, el flujo detiene su ejecución de forma inmediata.

Por otro lado en el nodo de HTTP se utilizó Continue para que la caída de la API no bloquee todo el flujo durante la ejecución y se desvíe a contingencia.

Error Workflow:

Este error representa un flujo global el cual se activa de manera automática ante cualquier error que se presente al instante. Sin este los flujos programados fallan en “silencio” sin notificar al programa.

Casos No Cubiertos:

A lo largo de este laboratorio hubieron casos que no se tocaron. Uno de estos fue el de modificar “sku”: “ZZ-999”, la limitación que presenta el flujo actual es que este evalúa las condiciones sobre montos, plazos y estado del producto, por lo que la variable sku no se considera y no se compara respecto a la base de datos. La solución a esto sería integrar un nodo de consulta previo al Switch el cual logre verificar la presencia del sku. Si el sku no retorna registros el sistema debe desviar la solicitud a la rama.

Diferencias entre URLs:

Dentro del nodo Webhook se presentaron 2 URL (test y production), el URL de Test está diseñado para la fase de construcción y depuración, esto permite visualizar el paso de los datos en tiempo real. Este URL requiere que la interfaz web permanezca abierta, además de necesitar que cada petición sea enviada de manera individual. Por otra parte el URL de producción está diseñado para la parte de operación asíncrona en segundo plano, este requiere que el flujo esté guardado y publicado. Este URL permite recibir peticiones de forma masiva y almacena el registro histórico en la pestaña de ejecuciones.

